

Ex. No.:10**Date:****Ensemble Learning – Random Forest, AdaBoost****Aim**

Implement the ensemble models to perform classification, and to evaluate the performance of the model.

Youtube video link: <https://youtu.be/pCtW0fRJPSo>

Description**Random Forest**

Random Forest is an ensemble learning technique that builds multiple decision trees and combines their outputs to improve classification accuracy and reduce overfitting. In this experiment, the dataset is loaded and preprocessed by handling missing values using mean imputation and converting categorical variables into numerical form using label encoding. The input features (X) and target variable (Y) are selected, and the dataset is split into training and testing sets.

Multiple Random Forest models are created by varying parameters such as the number of estimators (n_estimators) and maximum depth (max_depth). Each model is trained on the dataset, and predictions are made on the test data. The performance of each model is evaluated using metrics such as accuracy, precision, recall, and F1-score. A confusion matrix is also generated to analyze classification performance.

Key Steps

- Read dataset using pandas
- Handle missing values using mean imputation
- Perform label encoding for categorical variables
- Select input features (X) and target variable (Y)
- Split dataset into training and testing sets
- Define parameters (n_estimators, max_depth)
- Train multiple Random Forest models
- Predict class labels for test data
- Evaluate using accuracy, precision, recall, and F1-score
- Display confusion matrix

AdaBoost

AdaBoost (Adaptive Boosting) is an ensemble technique that combines multiple weak learners to form a strong classifier by focusing more on misclassified instances in each iteration. In this experiment, the dataset is preprocessed similarly by handling missing values and encoding categorical variables. The data is split into training and testing sets.

Multiple AdaBoost models are created by varying parameters such as the number of estimators

(n_estimators) and learning rate. Each model is trained iteratively, where misclassified samples are given higher importance. The trained models are evaluated using accuracy, precision, recall, and F1-score. A confusion matrix is used to further analyze classification results.

Key Steps

- Read dataset using pandas
- Handle missing values using mean imputation
- Perform label encoding
- Select input features (X) and target variable (Y)
- Split dataset into training and testing sets
- Define parameters (n_estimators, learning_rate)
- Train multiple AdaBoost models
- Predict class labels for test data
- Evaluate using accuracy, precision, recall, and F1-score
- Display confusion matrix

Q1. Random Forest

Program

```
print("URK23CS1092")
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
print("Libraries imported successfully")
```

Libraries imported successfully

1. Read CSV data into pandas dataframe object.

Code

```
print("URK23CS1092")
import pandas as pd
df = pd.read_csv("penguins.csv")
print("Dataset loaded successfully!\n")
print(df.head())
```

Output

Dataset loaded successfully!

```

species    island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen         39.1          18.7          181.0
1  Adelie  Torgersen         39.5          17.4          186.0
2  Adelie  Torgersen         40.3          18.0          195.0
3  Adelie  Torgersen         NaN           NaN           NaN
4  Adelie  Torgersen         36.7          19.3          193.0

body_mass_g  sex
0      3750.0  MALE
1      3800.0  FEMALE
2      3250.0  FEMALE
3         NaN   NaN
4      3450.0  FEMALE

```

2. Do necessary preprocessing [data imputation with mean for null values, label encoding to convert categorical to numerical].

Code

```

print("URK23CS1092")
print("\nStep 2: Data Preprocessing")
df.fillna(df.mean(numeric_only=True), inplace=True)
print("\nMissing values handled!\n")
le = LabelEncoder()
for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])
print("Categorical data encoded!\n")
print(df.head())

```

Output

```

Step 2: Data Preprocessing

Missing values handled!

Categorical data encoded!

species    island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0         0         2      39.10000      18.70000      181.00000
1         0         2      39.50000      17.40000      186.00000
2         0         2      40.30000      18.00000      195.00000
3         0         2      43.92193      17.15117      200.915205
4         0         2      36.70000      19.30000      193.00000

body_mass_g  sex
0  3750.00000    1
1  3800.00000    0
2  3250.00000    0
3  4201.754386    2
4  3450.00000    0

```

3. Choose independent variables (X1, X2) and dependent variable (Y) from givendataset

Code

```

print("URK23CS1092")
X = df.iloc[:, :-1]
Y = df.iloc[:, -1]
print("\nIndependent variables (X):\n", X.head())
print("\nDependent variable (Y):\n", Y.head())

```

Output

```

Independent variables (X):
  species  island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0         0        2         39.10000         18.70000         181.000000
1         0        2         39.50000         17.40000         186.000000
2         0        2         40.30000         18.00000         195.000000
3         0        2         43.92193         17.15117         200.915205
4         0        2         36.70000         19.30000         193.000000

  body_mass_g
0  3750.000000
1  3800.000000
2  3250.000000
3  4201.754386
4  3450.000000

Dependent variable (Y):
0  1
1  0
2  0
3  2
4  0
Name: sex, dtype: int32

```

4. Split the data into training and testing**Code**

```

print("URK23CS1092")
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42
)
print("\nData split completed!")
print("Training size:", X_train.shape)
print("Testing size:", X_test.shape)

```

Output

```

Data split completed!
Training size: (275, 6)
Testing size: (69, 6)

```

5. Assign the n_estimators [50, 100] and max_depth [2,3,4]**Code**

```

print("URK23CS1092")
n_estimators_list = [50, 100]
max_depth_list = [2, 3, 4]
print("\nParameters initialized!")
print("n_estimators:", n_estimators_list)
print("max_depth:", max_depth_list)

```

Output

```

Parameters initialized!
n_estimators: [50, 100]
max_depth: [2, 3, 4]

```

6. Create different models of Random Forest using different parameters.

7. For the different parameters of Random Forest, calculate the accuracy, precision, recall, and f1_score of the model

Code

```
print("URK23CS1092")
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
for n in n_estimators_list:
    for depth in max_depth_list:
        print("\n-----")
        print(f"Model with n_estimators={n}, max_depth={depth}")
        model = RandomForestClassifier(
            n_estimators=n,
            max_depth=depth,
            random_state=42
        )
        model.fit(X_train, Y_train)
        Y_pred = model.predict(X_test)
        acc = accuracy_score(Y_test, Y_pred)
        prec = precision_score(Y_test, Y_pred, average='weighted')
        rec = recall_score(Y_test, Y_pred, average='weighted')
        f1 = f1_score(Y_test, Y_pred, average='weighted')
        print("Accuracy:", acc)
        print("Precision:", prec)
        print("Recall:", rec)
        print("F1 Score:", f1)
```

Output

```
-----
Model with n_estimators=50, max_depth=2
Accuracy: 0.8260869565217391
Precision: 0.7903936545240893
Recall: 0.8260869565217391
F1 Score: 0.8077772867874503

-----
Model with n_estimators=50, max_depth=3
Accuracy: 0.8115942028985508
Precision: 0.7762604266627298
Recall: 0.8115942028985508
F1 Score: 0.7935119724143245

-----
Model with n_estimators=50, max_depth=4
Accuracy: 0.8115942028985508
Precision: 0.7770345596432553
Recall: 0.8115942028985508
F1 Score: 0.793623188405797

-----
Model with n_estimators=100, max_depth=2
```

```

Accuracy: 0.8115942028985508
Precision: 0.7762604266627298
Recall: 0.8115942028985508
F1 Score: 0.7935119724143245

-----

Model with n_estimators=100, max_depth=3
Accuracy: 0.8115942028985508
Precision: 0.7762604266627298
Recall: 0.8115942028985508
F1 Score: 0.7935119724143245

-----

Model with n_estimators=100, max_depth=4
Accuracy: 0.8260869565217391
Precision: 0.7904124860646599
Recall: 0.8260869565217391
F1 Score: 0.807526221884058

```

8. Display the confusion matrix

Code

```

print("URK23CS1092")
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(Y_test, Y_pred)
print("\nConfusion Matrix:\n")
print(cm)

```

Output

```

Confusion Matrix:

[[24  6  0]
 [ 8 28  0]
 [ 2  1  0]]

```

Q2. AdaBoost

Program

```

print("URK23CS1092")
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
print("Libraries imported successfully")

```

```
Libraries imported successfully
```

1. Read CSV data into pandas dataframe object

Code

```

print("URK23CS1092")
import pandas as pd

```

```
df = pd.read_csv("penguins.csv")
print("Dataset loaded successfully!\n")
print(df.head())
```

Output

```
Dataset loaded successfully!

  species  island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0  Adelie  Torgersen           39.1           18.7           181.0
1  Adelie  Torgersen           39.5           17.4           186.0
2  Adelie  Torgersen           40.3           18.0           195.0
3  Adelie  Torgersen           NaN           NaN           NaN
4  Adelie  Torgersen           36.7           19.3           193.0

  body_mass_g  sex
0      3750.0  MALE
1      3800.0  FEMALE
2      3250.0  FEMALE
3         NaN   NaN
4      3450.0  FEMALE
```

2. Do necessary preprocessing [data imputation with mean for null values, label encoding to convert categorical to numerical]

Code

```
print("URK23CS1092")
df.fillna(df.mean(numeric_only=True), inplace=True)
print("\nMissing values handled!\n")
le = LabelEncoder()
for col in df.select_dtypes(include='object').columns:
    df[col] = le.fit_transform(df[col])
print("Categorical data encoded!\n")
print(df.head())
```

Output

```
Missing values handled!

Categorical data encoded!

  species  island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0      0      2      39.10000      18.70000      181.00000
1      0      2      39.50000      17.40000      186.00000
2      0      2      40.30000      18.00000      195.00000
3      0      2      43.92193      17.15117      200.915205
4      0      2      36.70000      19.30000      193.00000

  body_mass_g  sex
0  3750.00000    1
1  3800.00000    0
2  3250.00000    0
3  4201.754386    2
4  3450.00000    0
```

3. Choose independent variables (X1, X2) and dependent variable (Y) from given dataset**Code**

```
print("URK23CS1092")
X = df.iloc[:, :-1]
Y = df.iloc[:, -1]
print("\nIndependent variables (X):\n", X.head())
print("\nDependent variable (Y):\n", Y.head())
```

Output

```
Independent variables (X):
  species  island  bill_length_mm  bill_depth_mm  flipper_length_mm  \
0        0        2        39.10000        18.70000        181.000000
1        0        2        39.50000        17.40000        186.000000
2        0        2        40.30000        18.00000        195.000000
3        0        2        43.92193        17.15117        200.915205
4        0        2        36.70000        19.30000        193.000000

  body_mass_g
0  3750.000000
1  3800.000000
2  3250.000000
3  4201.754386
4  3450.000000

Dependent variable (Y):
0    1
1    0
2    0
3    2
4    0
Name: sex, dtype: int32
```

4. Split the data into training and testing**Code**

```
print("URK23CS1092")
from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(
    X, Y, test_size=0.2, random_state=42
)
print("\nData split completed!")
print("Training size:", X_train.shape)
print("Testing size:", X_test.shape)
```

Output

```
Data split completed!
Training size: (275, 6)
Testing size: (69, 6)
```

5. Assign the n_estimators [50, 100] and learning_rate [0.01, 0.1, 1.0]**Code**

```
print("URK23CS1092")
n_estimators_list = [50, 100]
learning_rate_list = [0.01, 0.1, 1.0]
print("\nParameters initialized!")
print("n_estimators:", n_estimators_list)
```



```
print("learning_rate:", learning_rate_list)
```

Output

```
Parameters initialized!
n_estimators: [50, 100]
learning_rate: [0.01, 0.1, 1.0]
```

6. Create different models of AdaBoost using different parameters.

7. For the different parameters of AdaBoost, calculate the accuracy, precision, recall, and f1_score of the model

Code

```
print("URK23CS1092")
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score
for n in n_estimators_list:
    for lr in learning_rate_list:
        print("\n-----")
        print(f"Model with n_estimators={n}, learning_rate={lr}")
        model = AdaBoostClassifier(
            n_estimators=n,
            learning_rate=lr,
            random_state=42
        )
        model.fit(X_train, Y_train)
        Y_pred = model.predict(X_test)
        acc = accuracy_score(Y_test, Y_pred)
        prec = precision_score(Y_test, Y_pred, average='weighted')
        rec = recall_score(Y_test, Y_pred, average='weighted')
        f1 = f1_score(Y_test, Y_pred, average='weighted')
        print("Accuracy:", acc)
        print("Precision:", prec)
        print("Recall:", rec)
        print("F1 Score:", f1)
```

Output

```
-----
Model with n_estimators=50, learning_rate=0.01
Accuracy: 0.7246376811594203
Precision: 0.7009222661396574
Recall: 0.7246376811594203
F1 Score: 0.7094002405911669
-----
Model with n_estimators=50, learning_rate=0.1
```

```

Accuracy: 0.6521739130434783
Precision: 0.6642512077294686
Recall: 0.6521739130434783
F1 Score: 0.6318840579710144

-----

Model with n_estimators=50, learning_rate=1.0
Accuracy: 0.8695652173913043
Precision: 0.8321827262044653
Recall: 0.8695652173913043
F1 Score: 0.8503909777325213

-----

Model with n_estimators=100, learning_rate=0.01
Accuracy: 0.8115942028985508
Precision: 0.7796857873584215
Recall: 0.8115942028985508
F1 Score: 0.7941671769748928

-----

Model with n_estimators=100, learning_rate=0.1
Accuracy: 0.6666666666666666
Precision: 0.674308300395257
Recall: 0.6666666666666666
F1 Score: 0.6476469341758009

-----

Model with n_estimators=100, learning_rate=1.0
Accuracy: 0.7536231884057971
Precision: 0.724296675191816
Recall: 0.7536231884057971
F1 Score: 0.7375995101041029

```

8. Display the confusion matrix

Code

```

print("URK23CS1092")
from sklearn.metrics import confusion_matrix
cm = confusion_matrix(Y_test, Y_pred)
print("\nConfusion Matrix:\n")
print(cm)

```

Output

```

Confusion Matrix:

[[24  6  0]
 [ 8 28  0]
 [ 2  1  0]]

```

Result

The Random Forest and AdaBoost ensemble models are successfully implemented and evaluated, achieving high classification performance with improved accuracy and reliability.